

Conhecendo alguns modelos de Machine Learning

Definições

Conforme vimos nas aulas, quando temos um problema de regressão, o espaço de saída é da forma $\mathcal{Y} = \mathbb{R}$. Logo, para uma amostra $x \in \mathcal{X}$, uma função $h : \mathcal{X} \rightarrow \mathbb{R}$ pode ser usada diretamente para gerar $\hat{y} = h(x)$, que é uma predição. Note que estamos assumindo implicitamente que existe uma relação entre dados de entrada e saída.

Para encontrar a melhor curva, que represente essa relação, precisamos introduzir o conceito de **função de perda** (ou *loss function*), que pode ser definida como:

$$l(y, \hat{y}) : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$$

Observe que estamos interessados em encontrar uma expressão envolvendo y e $\hat{y}$. Uma possibilidade seria considerarmos, por exemplo, o erro absoluto:

$$l(y, \hat{y}) = |y - \hat{y}|$$

Outra abordagem seria utilizar o **erro quadrático**:

$$l(y, \hat{y}) = (y - \hat{y})^2$$

$l(y, \hat{y})$: é uma forma de modelar como o modelo está lidando com os dados.

Dada uma função $h : \mathcal{X} \rightarrow \mathcal{Y}$ qualquer, o erro “empírico” de h com respeito ao conjunto de observações D é dado por:

$$\mathcal{L}_D(h) = \frac{1}{N} \sum_{i=1}^N l(y_i, h(x_i))$$

Em Machine Learning, busca-se $h : \mathcal{X} \rightarrow \mathcal{Y}$ que aproxime a função-alvo f . Ou seja, busca-se h que minimize $\mathcal{L}_{\mathcal{X} \times \mathcal{Y}}(h)$ e para isso utiliza-se $\mathcal{L}_D(h)$. Ou seja, é uma função que se relaciona com os dados.

Para mais detalhes, sugiro consultar: Abu-Mostafa et al., 2012.

Regressão Linear

Resolve problema de regressão.

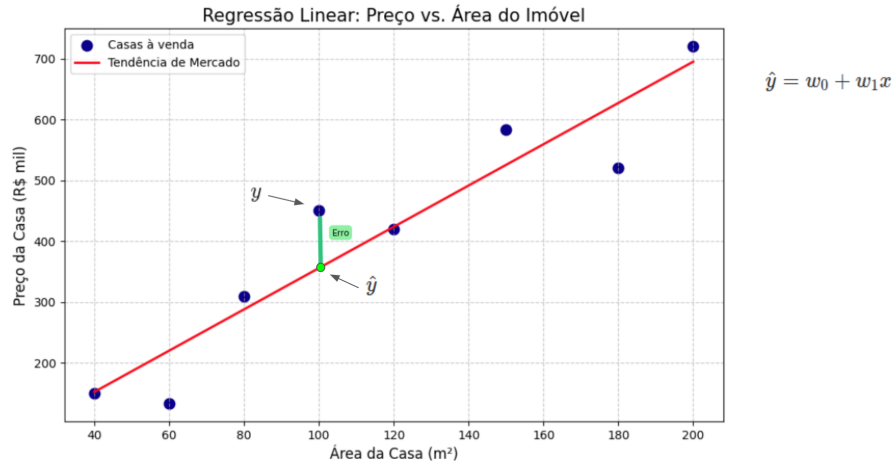


Figure 1: Exemplo usado em aula

Na regressão linear, temos: $l(y, \hat{y}) = (y - \hat{y})^2$

Ou seja, assumimos que o erro empírico (perda) de uma hipótese (preditor) $h \in \mathcal{H}$, com respeito aos dados D , é dado por:

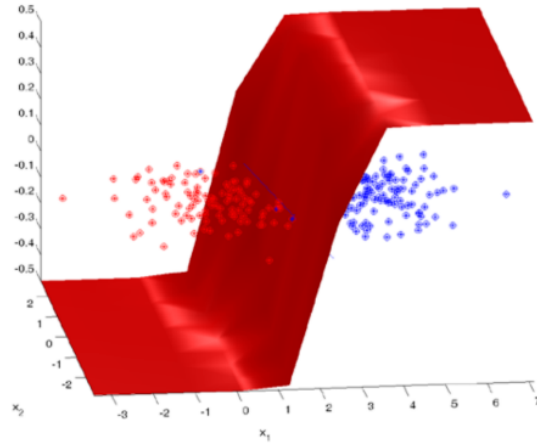
$$\mathcal{L}_D(h) = \frac{1}{N} \sum_{i=1}^N (y_i - h(x_i))^2$$

A regressão linear é um caso particular no qual a família de funções $\mathcal{H}$ é a família de funções lineares.

LinearRegression

Regressão Logística

Resolve problema de classificação.



Source: <http://strijov.com/sources/demoDataGen.php>

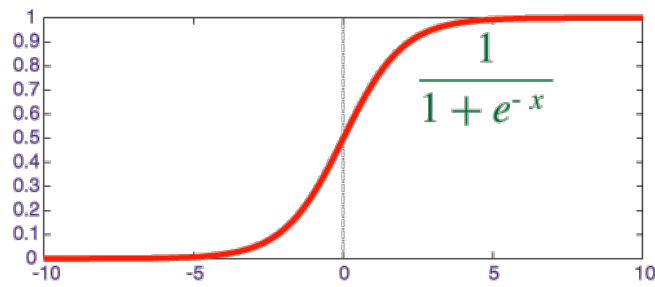


Figure 2: gabriel.mayers

Resolve o problema do “Sim” ou “Não” em termos de probabilidade. É ideal para problemas dicotômicos (classificação binária). Em vez de apenas informar se a classe é 0 ou 1, o modelo indica a probabilidade de o dado pertencer a cada uma delas. Para isso, utiliza uma curva chamada sigmoide.

LogisticRegression

SVM - Support Vector Machine (regressão e classificação)

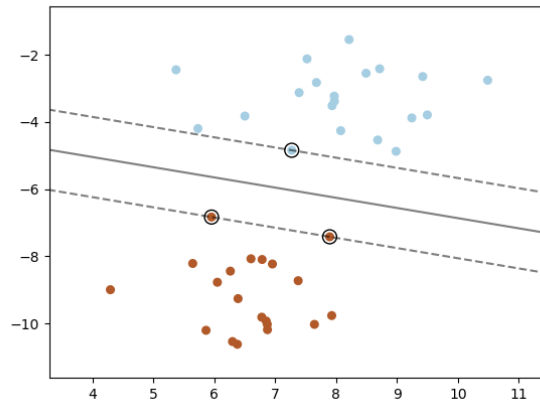


Figure 3: <https://scikit-learn.org/>

A ideia do modelo SVM é criar uma região, a mais ampla possível, para separar os pontos. Ele busca um “hiperplano” (uma fronteira) que deixe o maior espaço vazio possível entre as classes. O foco recai sobre os pontos mais difíceis (os que estão na fronteira), chamados de **vetores de suporte**.

Support Vector Machines

K-NN (regressão e classificação)

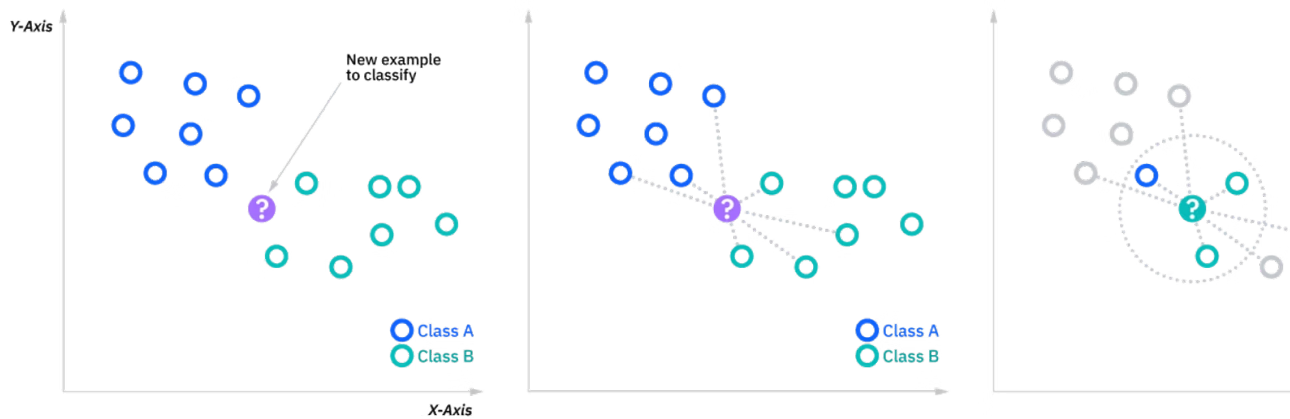


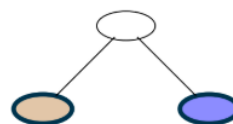
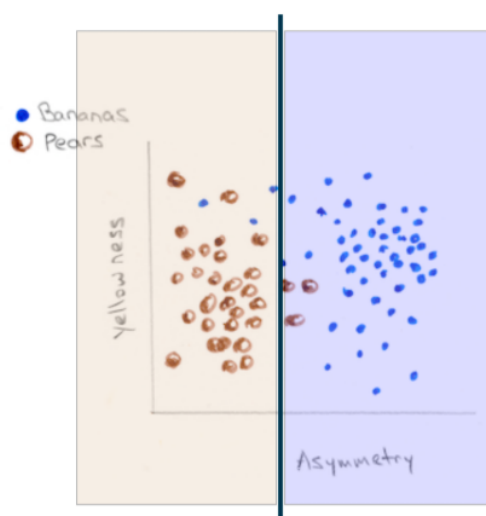
Figure 4: (Eda Kavlakoglu) IBM - machine Learning

É o algoritmo mais intuitivo. Ele se baseia na proximidade física (distância matemática) entre os dados. Para classificar um ponto novo, ele olha para os k vizinhos mais próximos. Se a maioria for da “Classe A”, o novo ponto será classificado como “Classe A”.

Esse modelo reforça a importância dos dados.

KNeighborsClassifier

Árvore de decisão (regressão e classificação)



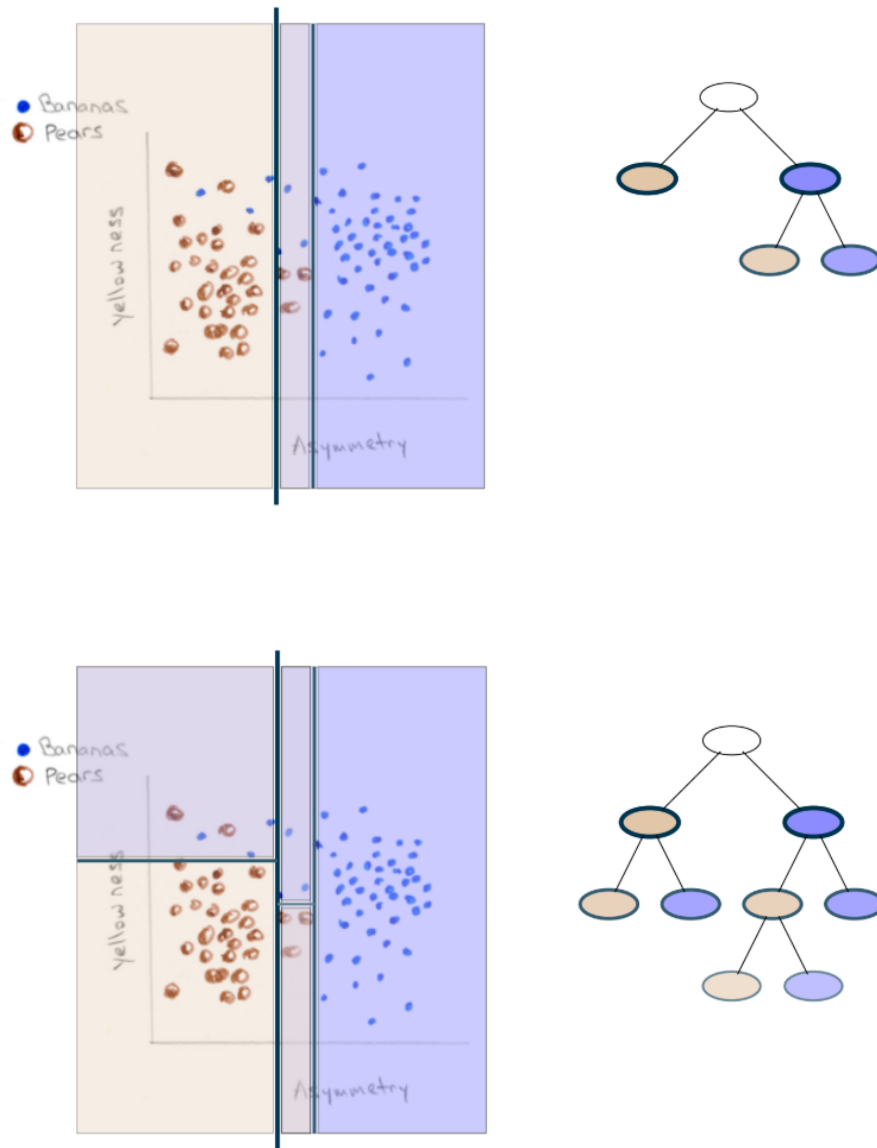


Figure 5: Duda et al

O algoritmo faz perguntas binárias sobre os dados (ex: “A fruta é redonda?”). Dependendo da resposta, ele segue por um caminho até chegar a uma decisão final. Isso quebra a complexidade do problema, tomando decisões simples ao longo do caminho.

TREE

Random Forest (regressão e classificação)

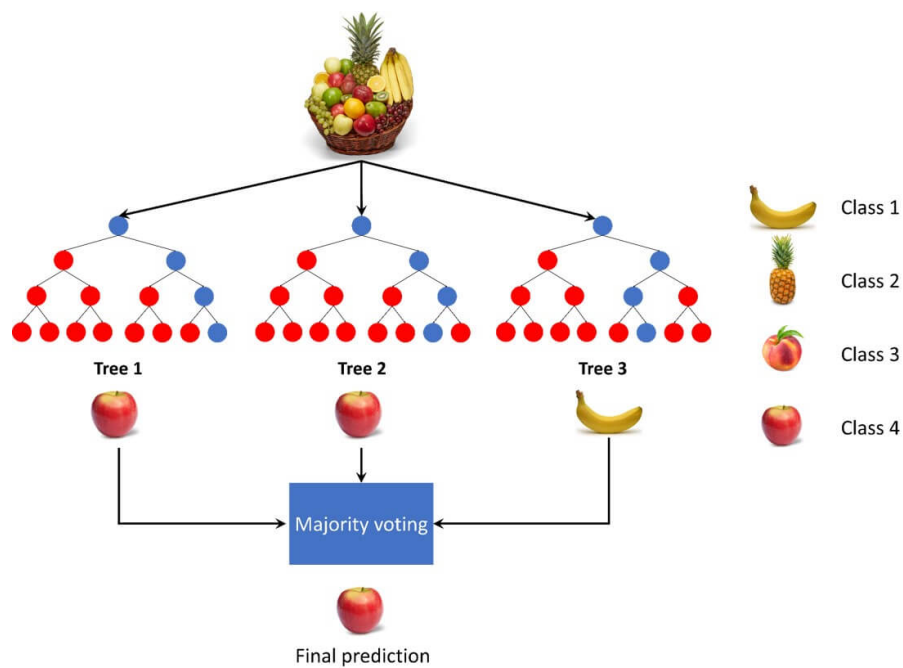


Figure 6: Strahinja Zivkovic

Uma árvore pode errar por ser muito específica. Utilizando centenas delas, é possível ter um certo controle sobre esse erro, aumentando a performance. Esse modelo cria várias árvores de decisão de forma aleatória e faz com que elas “votem”. O resultado final é a média (regressão) ou a maioria (classificação) dos votos.

RandomForestClassifier

XGBoost e LightGBM (regressão e classificação)

Em linhas gerais, esse modelo treina uma árvore simples e tenta entender onde ela está falhando. De posse dessa informação, ele treina uma próxima árvore focada, apenas, nos erros da primeira. Ele repete isso milhares de vezes e, assim, evolui de forma contínua através da correção de erros residuais.

São modelos de primeira linha, já me ajudaram a resolver problemas complexos.

GradientBoostingClassifier

LightGBM

XGBoost

Site: scikit-learn

<https://scikit-learn.org/stable/index.html>